```
      "title": "Elasticsearch Java"
    }
  }
}
```

A couple of things are going on here. We index a document into the same index (`programming_books3`) and then search for it. Because we canceled the refresh activity in listing 15.8, the `match` query doesn't yield any results.

Once indexing is complete, be sure to turn on refresh. (If refresh activity is disabled, no search queries will fetch the documents indexed during that refresh period.) We can reinstate the refresh interval by invoking this code.

Listing 15.10 Forcing a refresh on the index

```
POST programming_books3/_refresh
```

Invoking the `_refresh` endpoint at the end of the indexing operation (or periodically) ensures that indexed documents are available for searching. We can also allocate additional threads when working with bulk requests. To do so, we set the `thread_pool` attribute with additional writer threads in the elasticsearch.yml file, as the next listing shows.

Listing 15.11 Amending the writer threads' size

```
thread_pool:
  write:
    size: 50
    queue_size: 5000
```

Elasticsearch has many thread pools that we can change or modify according to our requirements. For example, in listing 15.11, the `write` (index) size of the pool is set to `50`. This means we have 50 threads to help index data in multiple threads. The `queue_size` attribute holds the requests in a queue until a thread is available to process the next request.

We can fetch the current thread pool by invoking this command: `GET _nodes/thread_pool`. It fetches the various thread pools created as part of the node configuration. The following snippet shows the `search` and `write` thread pool sizes when executing this command:

```
# GET _nodes/thread_pool
"thread_pool": {
  ....
  "search": {
    "type": "fixed",
    "size": 7,
    "queue_size": 1000
  },
```